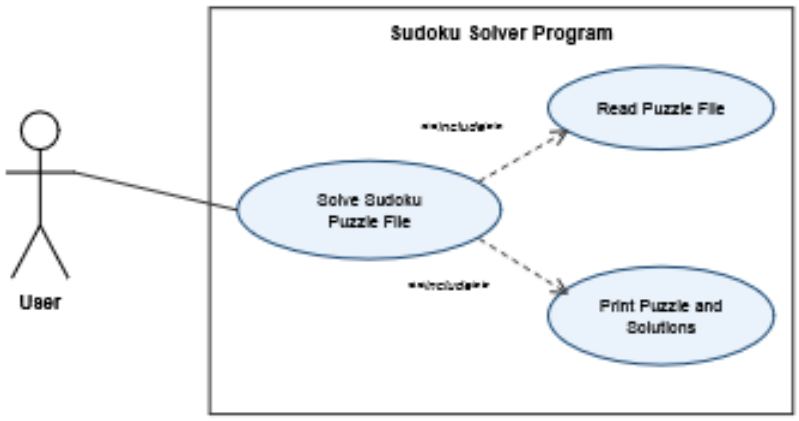
This use case model was prepared with the help of a GenAI (Claude, Anthropic).

Use Case Diagram



The user will run the program. Solving a puzzle file includes reading the file and printing the file and its solutions

a. **User Case Identified**
   - Primary actor: The user is the individual who runs the program. The user will provide 0 inputs. The program reads the five puzzle files automatically
   - Main Use Case: solving the Sudoku puzzle file
   - included use case:
     →Read puzzle file: load the 81-cell symbol profile into a 9 by 9 grid
     →Print puzzle and solutions: the program needs to print the file name, the original puzzle, and every solution or "no solution found."

b. **Brief description**
   - The user will begin by running the program, which reads each of the five puzzle files and is supposed to find every solution to the puzzles using recursive depth-first search with backtracking. Afterwards, it prints the file name, the original puzzle, and all solutions, or the text "no solutions found" when none exists.

c. **Step-by-step flow of events**
   - Main Flow

1. The user runs the program
2. The program starts with the first puzzle file
3. The program will open the file and read the 81 cell symbols into the 9 by 9 grid
4. Then the program prints the file name and the original puzzle
5. Next, the program finds the next blank cell and works out which digits are allowed there
6. The program has tried each allowed digit and repeats the search on the next empty cell
7. When the search ends, the program will print all solutions in total count or no solutions found
8. The program will repeat Step 2 to 8 for each remaining file
9. Program ends

- Alternative Flow and Exceptions
  - A1 - no solutions: when the program reaches step 8, no solution is found, so the program prints "No solution found."
  - A2 - multiple solutions: when the program reaches step 8, it will print every solution found, numbered, and the total
  - E1 - file missing or unreadable: when the program is at Step 3, the program prints an error message and moves to the next file
  - E2 - file malformed: when the program reaches Step 3, the file contains a character that is not 1 to 9, or it doesn't contain exactly 81 cells. Thus, the program will print an error message and move to the next file.
  - E3 - conflicting clues: if the given digits are repeated in a row, column, or block, the program finds no solution

d. Detailed Description

Use case name: Solve sudoku puzzle file

Actor: User

Goal: Obtain every solution, even if no solution exists, for each of the five sudoku puzzle files

Precautions: After the program has been fully compiled, puzzle one through puzzle 5 are in the same directory as the program. Each individual file holds 9 rows of nine cells, with each cell containing a digit (1-9) or an underscore for a blank.

Trigger: User runs the program

Main Flow:

1. The program will take the next puzzle file name
2. First, the program reads the file, ignoring spaces and line breaks, then builds a 9 by 9 grid.
3. Then the program prints the file name and the original puzzle
4. After that, it will move to the next empty cell and list the digits that do not already appear in the row, column, or three-by-three block
5. For each allowed digit, the program places it and repeats Step 4 on the next empty cell. However, if this cell has no allowed digit, the program will backtrack and determine the next digit for an earlier cell.
6. If no empty cells remain, the program saves the grid as the solution and will continue searching.
7. When the search is complete, the program prints each solution number and the total number found.
8. The program repeats from step one until all files are processed.

Alternative Flows:

- A1 – No solution: If nothing is saved after the search, the program prints “No solutions found.”
- A2 - multiple solutions: if the program has more than one grid saved, it prints all of them numbered, followed by the total

Exceptions:

- E1 - file missing or unreadable: the program will print “cannot open file (missing or unreadable)” then continues with the next file
- E2 - file malformed: the program will print an error describing the problem. This could be unexpected characters, too many cells, or too few cells. Then it will continue to the next file.

- E3 - conflicting clues: if the program reports “No solutions found” without searching because the given digits already break the rules

Post Conditions:

- For each file, the output will display the file name and either the original puzzle with all its solutions, a "No solutions found" message, or an error message. The program ends normally, and it does not change any input file.

Special requirements:

- The search is recursive and must not stop at the first solution. The program must be compiled and run on the EECS cycle servers.